

SESSION 6

From Training to Retrieval

Finishing the MLP — then building RAG

Karpathy MLP • Spring AI RAG



Two Halves, One Idea

Both halves are about the same thing: putting meaning into a vector space, and getting it back out.



Part One — Finishing the MLP

~20 minutes

- Finding a learning rate instead of guessing
- Train / dev / test, and what each one catches
- Plotting the embeddings — the payoff slide



Part Two — Retrieval (RAG)

the rest of the hour

- The two pipelines: indexing and query
- Keyword vs semantic vs metadata — three different jobs
- Chunking — the decision that quietly decides everything
- Why retrieval fails, live in the demo app

PART ONE

Finishing the MLP

Last time we stopped at minibatches. Three things left.



Stop Guessing the Learning Rate

0.1 was a guess. There is a five-minute experiment that replaces it.

1

Bracket it

Pick a range you're confident spans the answer — 0.001 up to 1.0.

2

Sweep exponentially

Step through the exponent, not the number. Equal spacing in log space.

3

Plot loss against it

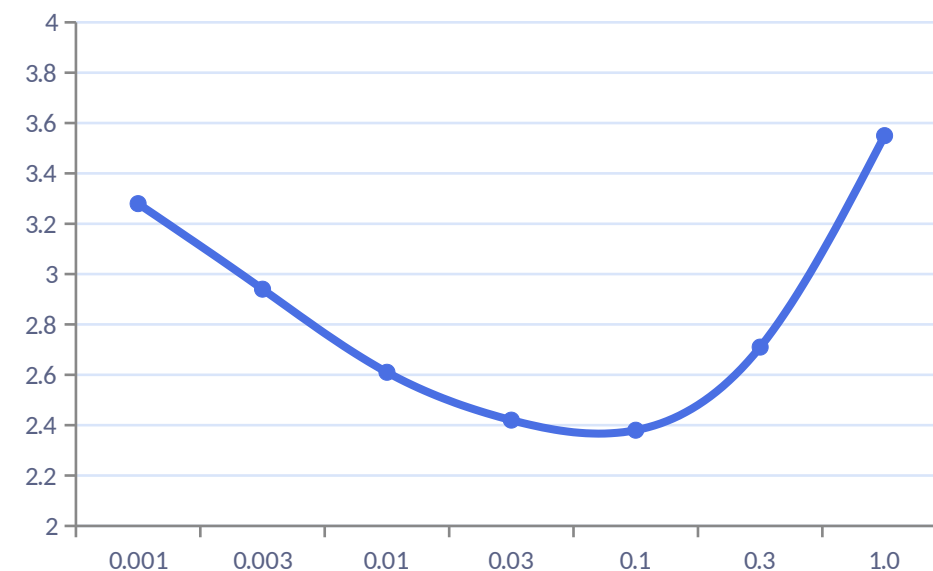
Run a few hundred steps per value and record where the loss lands.

4

Take the valley floor

Not the minimum point — the middle of the flat region before it climbs.

Loss vs learning rate



The flat valley is the answer — around 0.05 to 0.1 here.

Three Splits, Three Different Jobs

One split tells you nothing. The whole point is that each set answers a question the others can't.



~80%

TRAIN

Fits the parameters

Gradients come from here, and only here.



~10%

DEV

Chooses the hyperparameters

Layer size, embedding dimension, learning rate. Look at it often.



~10%

TEST

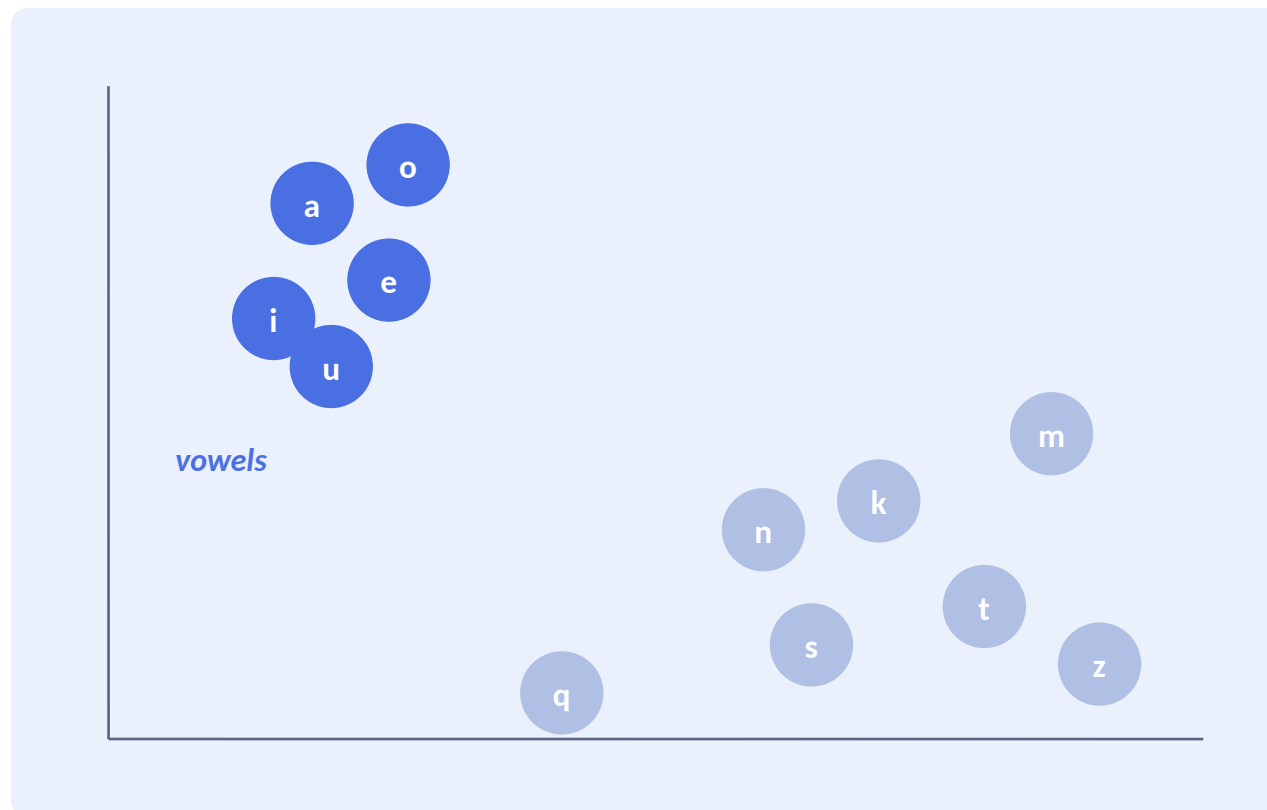
Reports the honest number

Touch it once, at the very end. Every peek burns it a little.

The tell: train loss and dev loss roughly equal means you are underfitting — the model is too small, not overtrained.

The Payoff: Look at the Embeddings

Nobody told it that vowels are a category. It worked that out from the loss alone.



Two numbers per character

The embedding dimension is 2 here purely so it can be drawn. Real models use hundreds.

Structure, not instruction

Vowels land together because they behave alike in the training data — nothing else.

This is nearest-neighbour search

Asking which characters sit closest is exactly what a vector database does, in 1,536 dimensions.

THE HANDOVER

You just trained an embedding table. Now you'll borrow one.

27 rows, 2 dimensions, learned from 200,000 names — against 30,000+ tokens and 1,536 dimensions, learned from the internet. The lookup is identical. Only the scale and the training objective changed.



PART TWO

Retrieval-Augmented Generation

How it works, why it breaks, and what to do about it.



Why RAG Exists At All

The model is excellent at language and knows nothing about your company. Those are separable problems.



It never saw your data

Internal docs, tickets, contracts, wikis. None of it was in the training set, and none of it ever will be.



Your data changes weekly

A policy updated on Monday should be answerable on Monday. Retraining cannot move at that speed.



You need to show the source

An answer nobody can verify is worthless in a business setting. Retrieval hands you the citation for free.

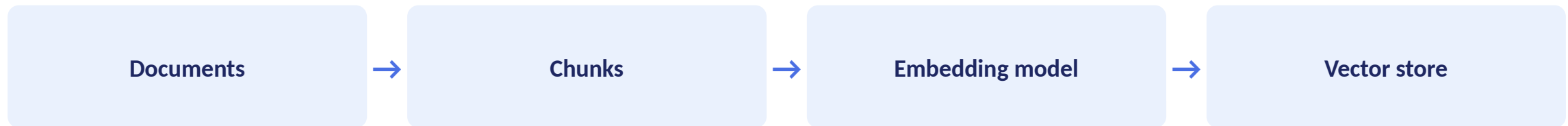
"Why not just fine-tune?"

Fine-tuning changes how the model writes — tone, format, structure. It is a poor and expensive way to teach it new facts, and it gives you no way to update or cite them. Retrieval changes what the model sees. Different tool, different problem.

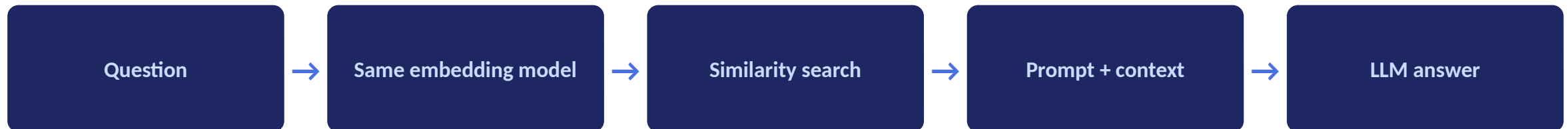
Two Pipelines, Not One

Almost every RAG bug is really a confusion about which of these two you are looking at.

INDEXING — runs once, ahead of time



QUERY — runs on every request



Both sides must use the same embedding model. Index with one and query with another and the distances are meaningless — you get results, they're just noise. This is the single most common setup mistake.

What Similarity Search Actually Does

There is no magic in the vector store. It measures angles and sorts.

1

Embed the question

The query becomes a vector in the same space as every stored chunk.

2

Measure cosine distance

The angle between two vectors. Close together means the model judged them to be about the same thing.

3

Return the top k

Sorted nearest first. You pick k — usually 3 to 5 for an answer, 20 if a reranker follows.

4

Paste them into the prompt

The chunks become context. The model reads them as if you had typed them yourself.



The quiet assumption

Every one of these steps assumes that "nearest in the embedding space" and "answers the question" are the same thing.

They usually overlap. They are not the same thing — and that gap is where every retrieval failure lives.

Three Ways to Find a Chunk

Everything so far has been semantic search. It is one of three mechanisms, and it is not always the right one.



Keyword search

Matches the words themselves

BM25 scores a chunk by how many of the query's rare terms it contains, and how often. No model, no vectors — just weighted counting.

Strong on: codes, IDs, error strings, names

Blind to: synonyms and paraphrase



Semantic search

Matches the meaning

Both sides go through the embedding model and you compare vectors. The words need not overlap at all — only the meaning has to.

Strong on: paraphrase and natural questions

Blind to: exact strings and numbers



Metadata filtering

Matches structured fields

Not a search at all. Each chunk carries fields you set at ingestion — year, section, author, type — and you restrict the candidates before ranking.

Strong on: scoping by date, source, permissions

Blind to: the content — it never reads the text

Which One Wins, and When

Read each query and ask what the retriever actually has to match on.

SEMANTIC	"How long do I have to send something back?"	Not one word of this appears in a policy titled Returns Window. Only the meaning connects them.
KEYWORD	"What causes ERR-409?"	The code is a rare string, not a concept. Embeddings blur it into every other error code.
METADATA	"What did the 2024 handbook say?"	Years embed badly. Filter on the field first, then search the smaller set.
HYBRID	"Warranty claim for model X-200 after 90 days"	A product code, a time condition, and a concept in one sentence. No single mechanism covers it.

Spring AI's VectorStore gives you similarity search and metadata filters — but no BM25. Hybrid means calling the store's native client, or running both queries and fusing the results yourself.

Chunking: Cutting Before You Embed

You cannot embed a 300-page manual as one vector. So you cut it — and whatever you cut becomes the unit you retrieve.



What a chunk is

- One passage of text, turned into one vector, stored as one row.
- Search returns whole chunks — never a sentence inside one.
- So a chunk has to be big enough to answer, and small enough to be about one thing.
- That tension is the entire problem. There is no setting that removes it.

Why it decides everything downstream

Retrieval quality

A perfect embedding model cannot rescue a chunk that was split mid-sentence.

Answer quality

The model only sees what you retrieved. Missing context becomes a confident wrong answer.

Cost and latency

Chunk size times k is your prompt size, on every single request.

Three Ways to Cut

There is no correct chunk size. It depends on how your documents are written — which makes it tuning, not configuration.



Fixed size

Every N tokens

Simple, predictable, and blind. It will cut sentences and tables in half without noticing.

Good default: 400–800 tokens



By structure

Headings, paragraphs, sections

Respects the author's own boundaries, so chunks are about one thing. Needs a document that has structure.

Best when the source is well-formed



Sliding window

Fixed size, with overlap

Each chunk repeats the tail of the last one, so a sentence near a boundary still carries its context.

Typical overlap: 10–20%

In practice you combine them: split on structure first, then cap anything oversized with a fixed splitter.

Where This Lives in Spring AI

Ingestion is a three-step ETL pipeline. Read, transform, write.

```
vectorStore.write( splitter.split( reader.read() ) );
```

DocumentReader

Reads the source into Documents

MarkdownDocumentReader for your .md file. PagePdfDocumentReader or ParagraphPdfDocumentReader for PDFs.

DocumentTransformer

Splits and enriches

TokenTextSplitter is the built-in fixed-size splitter. Structure-aware cutting or metadata enrichment is your own transformer.

DocumentWriter

Writes to the store

SimpleVectorStore keeps it all in memory — perfect for a demo, gone on restart.

Failure One: Similarity Is Not Relevance

The embedding measures whether two texts are about the same subject — not whether one answers the other.

THE QUESTION

"Can I return a damaged item?"

The user wants a yes or a no, and the conditions attached to it.

THE TOP HIT

"We do not accept returns on damaged items."

Same words, same topic, opposite meaning — and a near-perfect similarity score.



Opposites live close together in embedding space.

They have to — negation is a small part of a sentence, and the rest of it is identical. "We accept returns" and "we do not accept returns" are, to the model, two texts about returns. Nothing in the training objective taught it that one of them is the answer and the other is its exact opposite.

Failure Two: The Chunk Size Trap

Both directions fail, and they fail differently. That's why there's no safe default.

TOO SMALL

The answer loses its anchor

- The sentence with the answer gets separated from the line above that said which product it applied to.
- Retrieval finds a true statement and you have no idea what it's true about.
- Pronouns are the giveaway — "it must be returned within 14 days" is useless alone.

TOO BIG

The vector becomes an average

- A chunk covering five topics produces a vector that sits between all five.
- It ends up vaguely near every query and sharply near none, so it loses to more focused chunks.
- And when it does win, you've burned your context budget on four irrelevant topics.

Breaking the Demo On Purpose

Same app as last week. Markdown file, in-memory vector store, local model. This time we make it fail.

QUERY ONE

The exact term it can't find

Ask for a specific code — an error string, a product ID, a version number. Semantic search misses it, because a code carries almost no meaning to embed. Keyword search would have found it instantly.

Shows: embeddings are weak exactly where strings are precise

QUERY TWO

The confident wrong answer

Ask something the document answers in the negative, and watch the opposite-meaning chunk take the top slot. The model then answers fluently from the wrong context.

Shows: similarity is not relevance, live

Plant both cases in the markdown file beforehand, and run it once to confirm they actually break — the exact behaviour depends on the embedding model.

Three Fixes We Haven't Built Yet

Everything so far was the baseline. These are what you add when the baseline isn't good enough.



Reranking

Retrieve 20 cheaply, then score each one against the question with a smarter model and keep the best 3. Cheap to add, and it catches the similarity-isn't-relevance cases the embedding alone gets wrong.



Query rewriting

The user's question is vague, or full of pronouns from three messages ago. Rewrite it into a proper search query before you embed it — sometimes into several queries.



Contextual retrieval

Prepend a short summary of the surrounding document to each chunk before embedding it, so no chunk is stranded without the context that explains it.

Plus the big one: hybrid search — run keyword and semantic together, then merge the two ranked lists with reciprocal rank fusion.

What to Take Away

1

Find the learning rate, don't guess it

Sweep in log space and take the valley floor.

2

Three splits, three jobs

Train fits, dev chooses, test reports — once.

3

An embedding is a learned position

Whether you trained it or borrowed it, the lookup is the same.

4

Similarity is not relevance

Opposites are close together. Always.

5

Keyword, semantic, metadata

Three mechanisms, three different jobs.

6

Chunking is tuning, not configuration

The cut decides what you can retrieve.